

**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

Laboratório 2 de PSD

DFT

Curso: Engenharia de Telecomunicações
Professor: Elen Macedo Lobato

Aluno:
Victor E. de L. Guerra

13/11/2025

Sumário

1	Introdução	2
2	Roteiro e plots	2
2.1	Exemplo 1	2
2.2	Exemplo 2	2
2.3	Exemplo 3	3
2.4	Exemplo 4	4
2.5	Exemplo 5	5
2.6	Exemplo 6	6
2.7	Exemplo 7	7
2.8	Exemplo 8	8
2.9	Exemplo 9	9
2.10	Exemplo 10	10

1 Introdução

O estudo do processamento de sinais digital (PSD) envolve técnicas e ferramentas que permitem analisar, manipular e interpretar sinais em sua forma discreta. Entre estas ferramentas, destaca-se a *Transformada Discreta de Fourier* (DFT), que desempenha papel fundamental na análise espectral de sinais digitais.

A DFT possibilita a representação de um sinal no domínio da frequência a partir de sua descrição no tempo discreto. Em termos práticos, isso significa que podemos identificar quais componentes de frequência compõem determinado sinal, bem como suas respectivas amplitudes e fases. Esse tipo de análise é essencial em diversas aplicações, como compressão de áudio e imagem, comunicações digitais, reconhecimento de padrões e filtragem de sinais.

Neste laboratório, o objetivo é compreender o funcionamento da DFT e sua implementação computacional, seguindo o roteiro disponibilizado pela professora no SIGAA.

A realização deste experimento fornece uma base sólida para o entendimento das técnicas modernas de análise espectral e me prepara para lidar com problemas reais em engenharia e ciência de dados.

2 Roteiro e plots

2.1 Exemplo 1

O código abaixo calcula a *Transformada Discreta de Fourier* (DFT) de um vetor x utilizando o método direto:

```
1      x = [4 3 2 1];           % vetor de entrada
2      N = length(x);          % número de amostras
3      X = zeros(1,N);          % vetor para armazenar a DFT
4
5      for k = 0:N-1
6          for n = 0:N-1
7              X(k+1) = X(k+1) + x(n+1)*exp(-j*2*pi*n*k/N);
8          end
9      end
10
11     X1 = X;                   % resultado da DFT
12
13     %usando a fft
14     X2=fft(x);                % espaço reservado para outro cálculo ou comparação
15
```

$$X_1 = [10, 2 - 2j, 2, 2 + 2j]$$

$$X_2 = [10, 2 - 2j, 2, 2 + 2j]$$

2.2 Exemplo 2

O código a seguir gera um sinal impulso discreto de comprimento $N = 5$, calcula sua DFT utilizando a função `fft` e plota o sinal no domínio do tempo, bem como a magnitude e a fase da DFT.

```

1 N=5;
2 n=0:N-1;
3 x=[1 0 0 0 0];
4 figure(1)
5 subplot(311);
6 stem(n,x)
7 title('Sinal x[n] - impulso');
8 k=0:N-1;
9 X=fft(x);
10 subplot(312);
11 stem(k,abs(X));
12 title('Magnitude da DFT');
13 subplot(313);
14 stem(k,angle(X))
15 title('Fase da DFT');

```

Figura 1: Código do exemplo 2

E abaixo podemos ver o plot que esse código realiza:

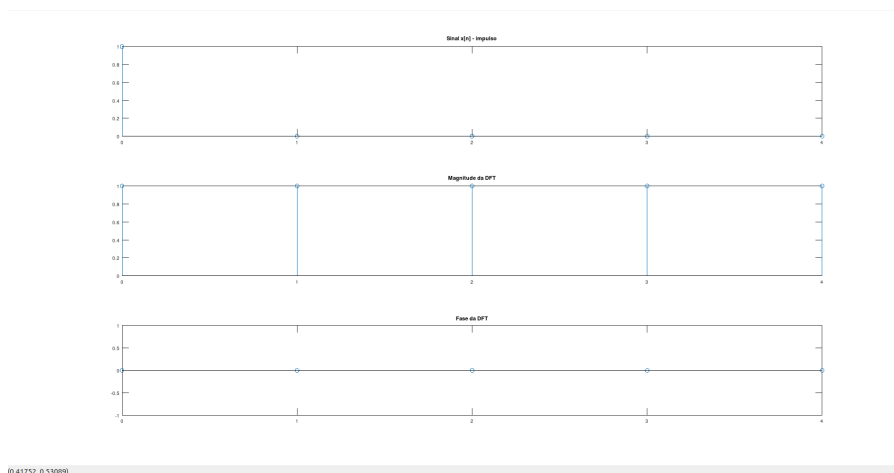


Figura 2: Plot do exemplo 2

2.3 Exemplo 3

O código a seguir gera um sinal constante de comprimento $N = 50$, calcula sua DFT utilizando a função `fft` e plota o sinal no domínio do tempo, bem como a magnitude e a fase da DFT.

```

1 N=50;
2 n=0:N-1;
3 x=ones(1,N);
4 figure(2)
5 subplot(311);
6 stem(n,x)
7 title('Sinal x[n] - constante');
8 k=0:N-1;
9 X=fft(x,N);
10 subplot(312);
11 stem(k,abs(X))
12 title('Magnitude da DFT');
13 subplot(313);
14 stem(k,angle(X))
15 title('Fase da DFT');

```

Figura 3: Código do exemplo 3

E abaixo podemos ver o plot que esse código realiza:

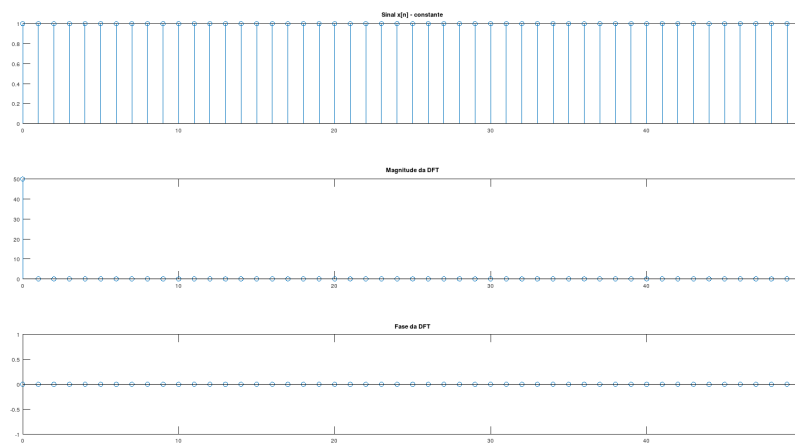


Figura 4: Plot do exemplo 3

2.4 Exemplo 4

O código a seguir gera um sinal discreto de comprimento $N = 5$, calcula sua DFT utilizando a função `fft` e plota o sinal no domínio do tempo, bem como a magnitude e a fase da DFT.

```

1 N=5;
2 n=0:N-1;
3 x=[1 1 1 0 0];
4 figure(3)
5 subplot(311);
6 stem(n,x)
7 title('Sinal x[n]');
8 k=0:N-1;
9 X=fft(x,N);
10 subplot(312);
11 stem(k,abs(X))
12 title('Magnitude da DFT');
13 subplot(313);
14 stem(k,angle(X))
15 title('Fase da DFT');

```

Figura 5: Código do exemplo 4

E abaixo podemos ver o plot que esse código realiza:

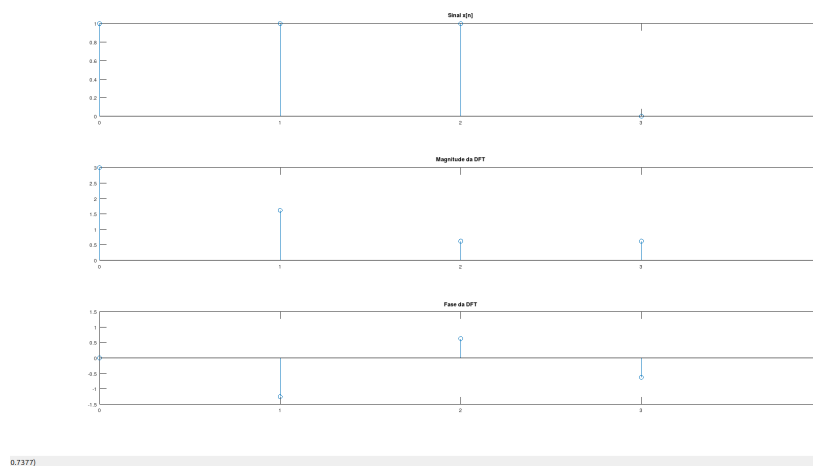


Figura 6: Plot do exemplo 4

2.5 Exemplo 5

O código a seguir gera um sinal discreto de comprimento $N = 10$, contendo valores positivos e negativos, calcula sua DFT utilizando a função `fft` e plota o sinal no domínio do tempo, bem como a magnitude e a fase da DFT.

```

1 N=10;
2 n=0:N-1;
3 x=[1 1 1 1 1 -1 -1 -1 -1 -1];
4 figure(4)
5 subplot(311);
6 stem(n,x)
7 title('Sinal x[n]');
8 k=0:N-1;
9 X=fft(x);
10 subplot(312);
11 stem(k,abs(X))
12 title('Magnitude da DFT');
13 subplot(313);
14 stem(k,angle(X))
15 title('Fase da DFT');

```

Figura 7: Código do exemplo 5

E abaixo podemos ver o plot que esse código realiza:

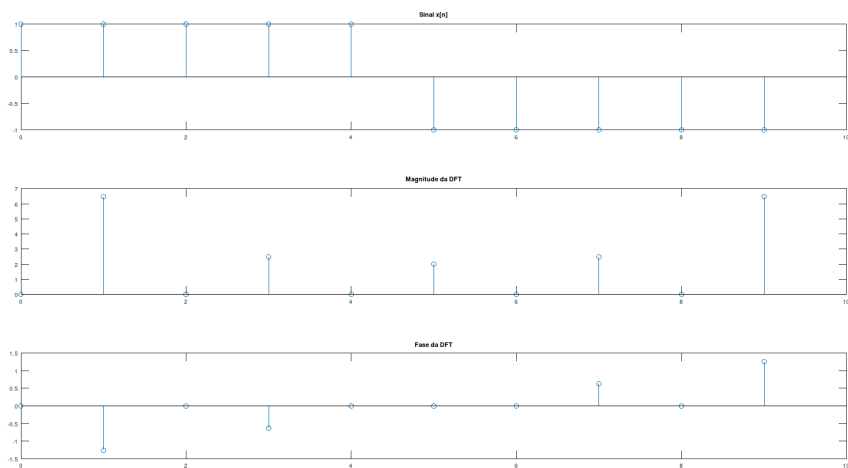


Figura 8: Plot do exemplo 5

2.6 Exemplo 6

O código a seguir calcula a DFT de um sinal, aplica um deslocamento circular no domínio da frequência e retorna ao domínio do tempo usando a IFFT.

```

1 N=6;
2 k=0:N-1;
3 n=0:N-1;
4 x=[4 3 2 1 0 0];
5 figure(5)
6 subplot(211);
7 stem(n,x)
8 title('Sinal x[n]');
9 X=fft(x);
10 Y=exp(j*2*pi*4*k/6).*X;
11 y=ifft(Y);
12 subplot(212);
13 stem(n,abs(y));
14 title('Sinal y[n]');

```

Figura 9: Código do exemplo 6

E abaixo podemos ver o plot que esse código realiza:

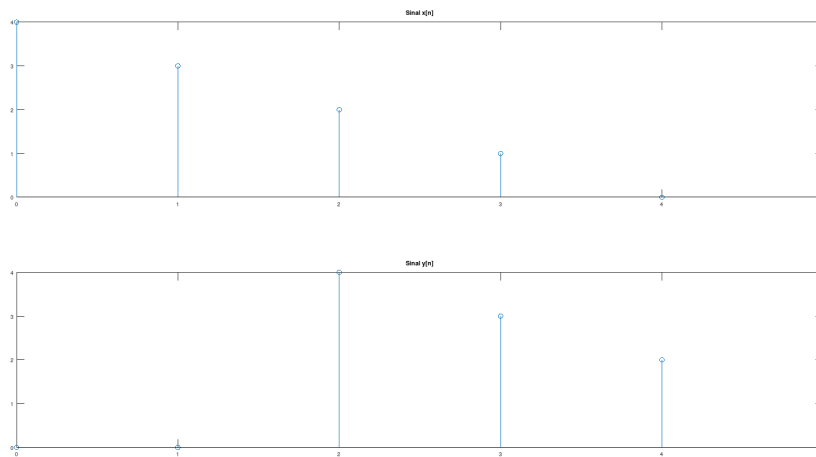


Figura 10: Plot do exemplo 6

2.7 Exemplo 7

O código a seguir calcula a DFT de um sinal, realiza a convolução circular no domínio da frequência e compara com a convolução circular direta usando `cconv`.

```

1 N=5;
2 n=0:N-1;
3 x=[2 1 0 1 0];
4 figure(6)
5 subplot(311);
6 stem(n,x);
7 title('Sinal x[n]');
8 X=fft(x);
9 Y=X.^2;
10 y=ifft(Y);
11 subplot(312);
12 stem(n,y);
13 title('Sinal y[n] pela DFT');
14 subplot(313);
15 stem(n,cconv(x,x,5));
16 title('Sinal y[n] pela convolução');

```

Figura 11: Código do exemplo 7

E abaixo podemos ver o plot que esse código realiza:

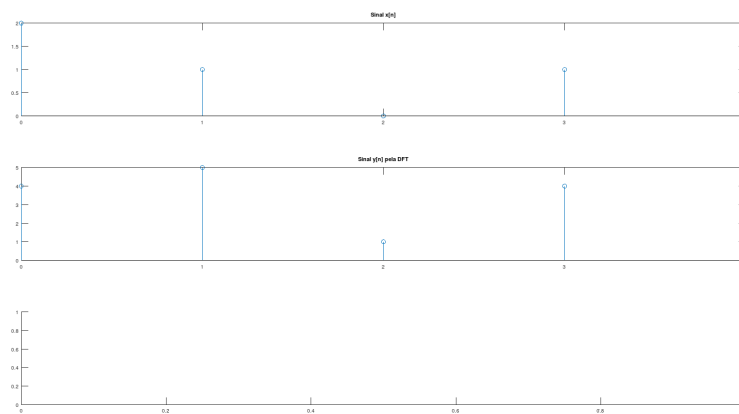


Figura 12: Plot do exemplo 7

2.8 Exemplo 8

O código a seguir calcula a DFT de dois sinais, realiza a convolução circular no domínio da frequência e compara com a convolução circular direta usando `cconv`.

```

1 N=5;
2 n=0:N-1;
3 x=[1 3 3 2 0];
4 h=[1 1 1 1 0];
5 figure(7)
6 subplot(411);
7 stem(n,x)
8 title('Sinal x[n]');
9 subplot(412);
10 stem(n,h)
11 title('Sinal h[n]');
12 X=fft(x);
13 H=fft(h);
14 Y=X.*H;
15 y=ifft(Y);
16 subplot(413);
17 stem(n,y);
18 title('Sinal y[n] obtido pela DFT');
19 subplot(414);
20 stem(n,cconv(x,h,5));
21 title('Sinal y[n] obtido pela convolução');

```

Figura 13: Código do exemplo 8

E abaixo podemos ver o plot que esse código realiza:

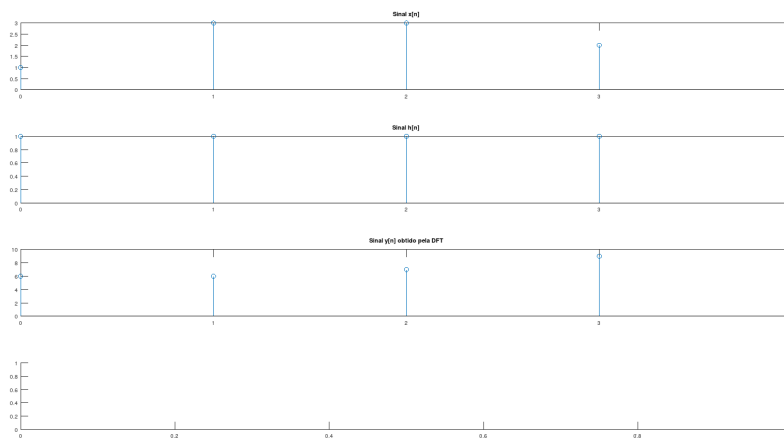


Figura 14: Plot do exemplo 8

2.9 Exemplo 9

O código a seguir calcula a DFT de um sinal, multiplica pelo conjugado para obter a autocorrelação no domínio da frequência e retorna ao domínio do tempo usando IFFT.

```

1 N=10;
2 n=0:N-1;
3 x=[1 1 1 1 1 1 0 0 0 0];
4 figure(8)
5 subplot(211);
6 stem(n,x);
7 title('Sinal x[n]');
8 X=fft(x);
9 Y=X.*conj(X);
10 y=ifft(Y);
11 subplot(212);
12 stem(n,y);
13 title('Sinal y[n]');

```

Figura 15: Código do exemplo 9

E abaixo podemos ver o plot que esse código realiza:

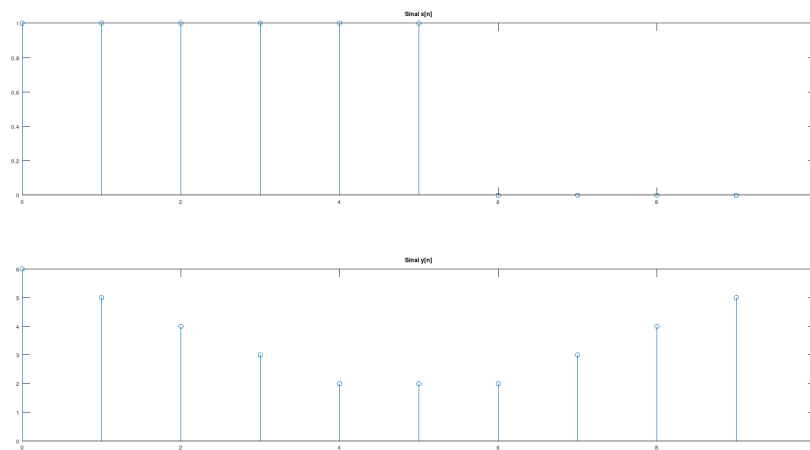


Figura 16: Plot do exemplo 9

2.10 Exemplo 10

O código a seguir gera dois sinais senoidais de diferentes amplitudes e frequências, adiciona ruído gaussiano e plota o sinal limpo no domínio do tempo.

```

1 amplitude_1 = 5;
2 freq_1 = 5;
3 amplitude_2 = 2;
4 freq_2 = 50;
5 Fs = 1000;
6 time = 0:1/Fs:(1-1/Fs);
7 sine_1 = amplitude_1*sin(2*pi*freq_1.*time);
8 sine_2 = amplitude_2*sin(2*pi*freq_2.*time);
9 noise = randn(1,length(time));
10 x_clean = sine_1 + sine_2;
11 x_noisy = x_clean + noise;
12 figure(9);
13 plot(time,x_clean);

```

Figura 17: Código do exemplo 10

E abaixo podemos ver o plot que esse código realiza:

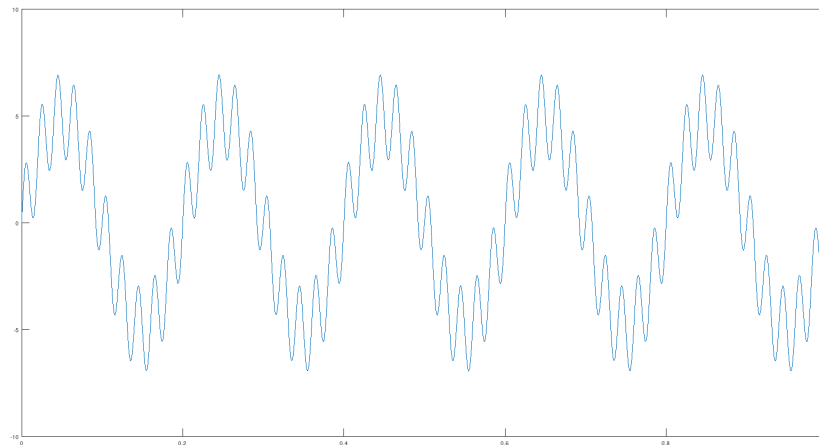


Figura 18: Plot do exemplo 10

Com alteração da linha de plot (linhas 12 e 13) para:

```
1 figure(10);  
2 plot(time,x_noisy);  
3
```

Temos o seguinte plot:

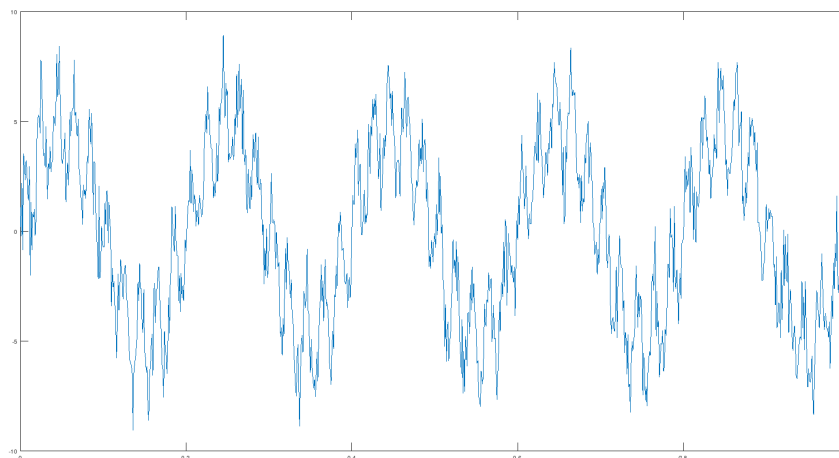


Figura 19: Plot do exemplo 10 com alteração do plot, utilizando um x com ruído

O código a seguir calcula a FFT do sinal senoidal limpo, normaliza a amplitude, realiza a centragem da transformada e plota o espectro no domínio da frequência.

```
1      XC = fft(x_clean);  
2      L=length(XC);  
3      XC = abs(2*XC/L);  
4      XC=fftshift(XC);  
5      freq = Fs*(-(L/2):(L/2)-1)/L;  
6      plot(freq,XC)  
7      axis([-100 100 -1 6])  
8
```

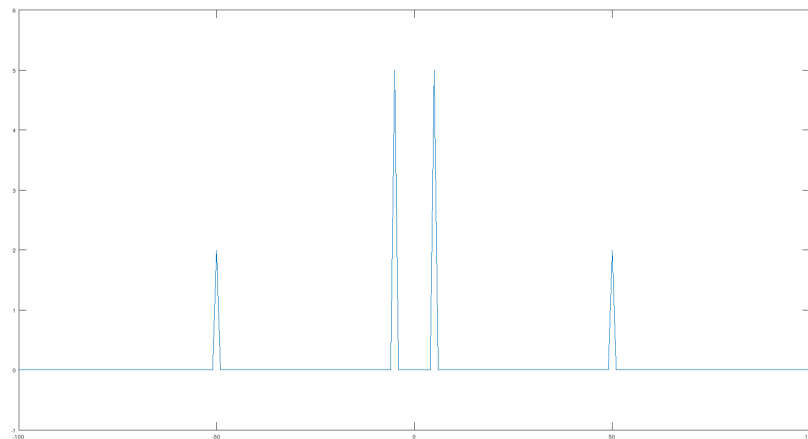


Figura 20: Espectro do sinal limpo normalizado no domínio da frequência

O código a seguir calcula a FFT do sinal com ruído, normaliza a amplitude, realiza a centragem da transformada e plota o espectro no domínio da frequência.

```
1      XN = fft(x_noisy);  
2      XN = abs(2*XN/L);  
3      XN=fftshift(XN);  
4      freq = Fs*(-(L/2):(L/2)-1)/L;  
5      plot(freq,XN)  
6      axis([-100 100 -1 6])  
7
```

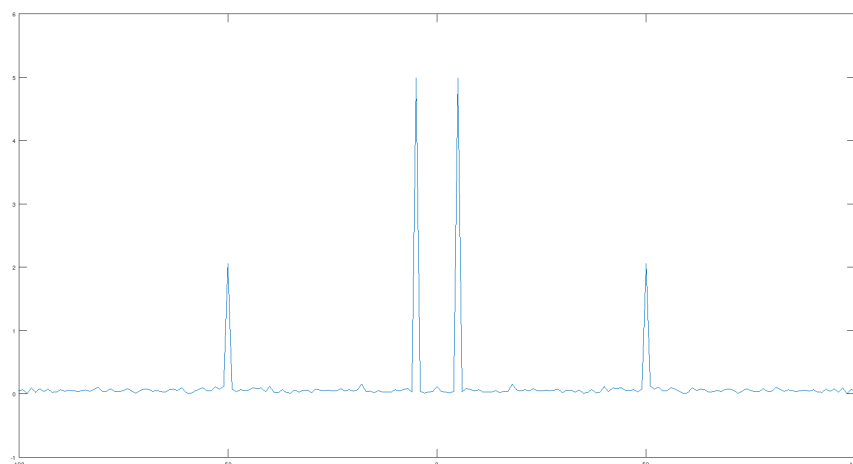


Figura 21: Espectro do sinal com ruído normalizado no domínio da frequência

O código a seguir aplica um filtro médio móvel ao sinal com ruído usando `conv` e `filter`, calcula a FFT do sinal filtrado e plota o espectro.

```
1      N=10;
2      b = ones(1,N);
3      y2 = conv(x_noisy,b/N);
4      y1 = filter(b,N,x_noisy);
5      Y = fft(y2);
6      L=length(Y);
7      Y = abs(2*Y/L);
8      Y=fftshift(Y);
9      freq = Fs*(-(L/2):(L/2)-1)/L;
10     plot(freq,Y)
11     axis([-100 100 -1 6])
12
```

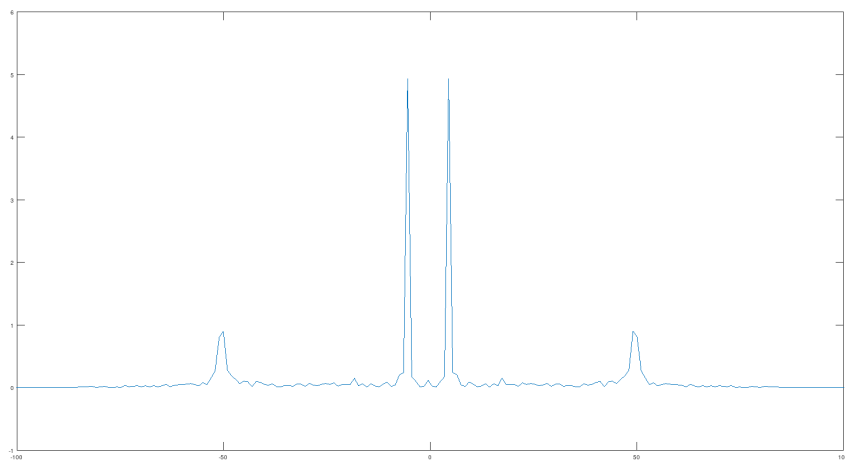


Figura 22: Espectro do sinal após ser filtrado